Experiment 1

Title

Compare Virtual Machine and Container

Objective

To study Virtual Machines and Containers, deploy services using both approaches, analyze system resource usage, and conclude which virtualization technique is more efficient.

Tools Used

- Oracle VirtualBox
- Vagrant
- Ubuntu (Jammy 22.04)
- Nginx
- Docker
- Windows Task Manager

Theory

Virtual Machine (VM) and Container are two different approaches to virtualization.

A Virtual Machine provides hardware-level virtualization and runs a complete operating system. Each VM includes its own OS, libraries, and dependencies. Due to this, Virtual Machines consume more CPU, memory, and storage resources. VM startup time is also higher.

Containers provide operating system-level virtualization. Containers share the host operating system kernel and only package the application with its dependencies. This makes containers lightweight, faster, and more resource-efficient.

In this experiment, a Virtual Machine is created using VirtualBox and Vagrant, and Nginx is installed inside it. Later, the same service is deployed using Docker containers. Resource usage is compared to determine which approach performs better.

Procedure

Part A: Virtual Machine Setup

Step 1: Install VirtualBox

Oracle VirtualBox was installed to manage virtual machines.

Step 2: Install Vagrant

Vagrant was installed to automate VM creation.

```
pratiksmac@MacBook-3 ~ % brew install --cask vagrant
✔ JSON API cask.jws.json                        Downloaded   15.4MB/ 15.4MB
✔ JSON API formula.jws.json                     Downloaded   32.0MB/ 32.0MB
==> Fetching downloads for: vagrant
✔ Cask vagrant (2.4.9)                          Verified     56.2MB/ 56.2MB
==> Installing Cask vagrant
==> Running installer for vagrant with `sudo` (which may request your password).
Password:
installer: Package name is Vagrant
installer: Installing at base path /
installer: The install was successful.
==> Linking Bash Completion 'completion.sh' to '/opt/homebrew/etc/bash_completio
==> Linking Zsh Completion '_vagrant' to '/opt/homebrew/share/zsh/site-functions
🍺  vagrant was successfully installed!
pratiksmac@MacBook-3 ~ % vagrant --version
Vagrant 2.4.9
pratiksmac@MacBook-3 ~ %
```

```
pratiksmac@MacBook-3 Experiment-1 % vagrant plugin install vagrant-docker-compose
Installing the 'vagrant-docker-compose' plugin. This can take a few minutes...
Fetching vagrant-docker-compose-1.5.1.gem
Installed the plugin 'vagrant-docker-compose (1.5.1)'!
pratiksmac@MacBook-3 Experiment-1 % vagrant up --provider=docker
Bringing machine 'default' up with 'docker' provider...
==> default: Box 'ubuntu/jammy64' could not be found. Attempting to find and install...
    default: Box Provider: docker
    default: Box Version: >= 0
==> default: Loading metadata for box 'ubuntu/jammy64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
The box you're attempting to add doesn't support the provider
you requested. Please find an alternate box or use an alternate
provider. Double-check your requested provider to verify you didn't
simply misspell it.

If you're adding a box from HashiCorp's Vagrant Public Registry, make sure the box is
released.

Name: ubuntu/jammy64
Address: https://vagrantcloud.com/api/v2/vagrant/ubuntu/jammy64
Requested provider: docker
pratiksmac@MacBook-3 Experiment-1 %
```

Step 3: Verify Vagrant Installation

```
vagrant --version
```

```
pratiksmac@MacBook-3 ~ % qemu-system-aarch64 --version
QEMU emulator version 11.0.0
Copyright (c) 2003-2026 Fabrice Bellard and the QEMU Project developers
pratiksmac@MacBook-3 ~ % vagrant plugin install vagrant-qemu
Installing the 'vagrant-qemu' plugin. This can take a few minutes...
Fetching vagrant-qemu-0.3.12.gem
Installed the plugin 'vagrant-qemu (0.3.12)'!
pratiksmac@MacBook-3 ~ % vagrant plugin list
vagrant-docker-compose (1.5.1, global)
vagrant-qemu (0.3.12, global)
pratiksmac@MacBook-3 ~ %
```

Step 4: Initialize Ubuntu Virtual Machine

```
vagrant init ubuntu/jammy64
```

```
pratiksmac@MacBook-3 Experiment-1 % vagrant up --provider=qemu
Bringing machine 'default' up with 'qemu' provider...
==> default: Box 'perk/ubuntu-2204-arm64' could not be found. Attempting to find and install...
    default: Box Provider: libvirt
    default: Box Version: >= 0
==> default: Loading metadata for box 'perk/ubuntu-2204-arm64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/perk/ubuntu-2204-arm64
==> default: Adding box 'perk/ubuntu-2204-arm64' (v20250702) for provider: libvirt (arm64)
    default: Downloading: https://vagrantcloud.com/perk/boxes/ubuntu-2204-arm64/versions/20250702/providers/libvirt/arm64/vagrant.box
    default: Calculating and comparing box checksum...
==> default: Successfully added box 'perk/ubuntu-2204-arm64' (v20250702) for 'libvirt (arm64)'!
==> default: Checking if box 'perk/ubuntu-2204-arm64' version '20250702' is up to date...
==> default: Importing a QEMU instance
    default: Creating and registering the VM...
    default: Successfully imported VM
==> default: Warning! The QEMU provider doesn't support any of the Vagrant
==> default: high-level network configurations (`config.vm.network`). They
==> default: will be silently ignored.
==> default: Starting the instance...
==> default: Waiting for machine to boot. This may take a few minutes...
    default: SSH address: 127.0.0.1:50022
    default: SSH username: vagrant
    default: SSH auth method: private key
```

Step 5: Start the Virtual Machine

```
vagrant up
```

```
pratiksmac@MacBook-3 Experiment-1 % vagrant up --provider=qemu
Bringing machine 'default' up with 'qemu' provider...
==> default: Checking if box 'perk/ubuntu-2204-arm64' version '20250702' is up to date...
==> default: Machine already provisioned. Run `vagrant provision` or use the `--provision`
==> default: flag to force provisioning. Provisioners marked to run always will still run.
pratiksmac@MacBook-3 Experiment-1 %
```

Step 6: Access Ubuntu VM

```
vagrant ssh
```

```
Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


Last login: Fri Jul 11 14:08:27 2025
vagrant@ubuntu:~$ sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
curl localhost
Hit:1 http://ports.ubuntu.com/ubuntu-ports jammy InRelease
Get:2 http://ports.ubuntu.com/ubuntu-ports jammy-updates InRelease [128 kB]
Get:3 http://ports.ubuntu.com/ubuntu-ports jammy-backports InRelease [127 kB]
Get:4 http://ports.ubuntu.com/ubuntu-ports jammy-security InRelease [129 kB]
Get:5 http://ports.ubuntu.com/ubuntu-ports jammy/universe arm64 Packages [13.9 MB]
Get:6 http://ports.ubuntu.com/ubuntu-ports jammy/universe Translation-en [5652 kB]
Get:7 http://ports.ubuntu.com/ubuntu-ports jammy/universe arm64 c-n-f Metadata [277 kB]
Get:8 http://ports.ubuntu.com/ubuntu-ports jammy/multiverse arm64 Packages [184 kB]
Get:9 http://ports.ubuntu.com/ubuntu-ports jammy/multiverse Translation-en [112 kB]
Get:10 http://ports.ubuntu.com/ubuntu-ports jammy/multiverse arm64 c-n-f Metadata [7064 B]
Get:11 http://ports.ubuntu.com/ubuntu-ports jammy-updates/main arm64 Packages [3323 kB]
Get:12 http://ports.ubuntu.com/ubuntu-ports jammy-updates/main Translation-en [523 kB]
Get:13 http://ports.ubuntu.com/ubuntu-ports jammy-updates/main arm64 c-n-f Metadata [19.4 kB]
Get:14 http://ports.ubuntu.com/ubuntu-ports jammy-updates/restricted arm64 Packages [5947 kB]
Get:15 http://ports.ubuntu.com/ubuntu-ports jammy-updates/restricted Translation-en [1119 kB]
Get:16 http://ports.ubuntu.com/ubuntu-ports jammy-updates/restricted arm64 c-n-f Metadata [448 B]
Get:17 http://ports.ubuntu.com/ubuntu-ports jammy-updates/universe arm64 Packages [1318 kB]
Get:18 http://ports.ubuntu.com/ubuntu-ports jammy-updates/universe Translation-en [316 kB]
Get:19 http://ports.ubuntu.com/ubuntu-ports jammy-updates/universe arm64 c-n-f Metadata [27.3 kB]
Get:20 http://ports.ubuntu.com/ubuntu-ports jammy-updates/multiverse arm64 Packages [52.0 kB]
Get:21 http://ports.ubuntu.com/ubuntu-ports jammy-updates/multiverse Translation-en [15.5 kB]
Get:22 http://ports.ubuntu.com/ubuntu-ports jammy-updates/multiverse arm64 c-n-f Metadata [488 B]
Get:23 http://ports.ubuntu.com/ubuntu-ports jammy-backports/main arm64 Packages [94.1 kB]
Get:24 http://ports.ubuntu.com/ubuntu-ports jammy-backports/main Translation-en [11.4 kB]
Get:25 http://ports.ubuntu.com/ubuntu-ports jammy-backports/main arm64 c-n-f Metadata [416 B]
Get:26 http://ports.ubuntu.com/ubuntu-ports jammy-backports/restricted arm64 c-n-f Metadata [116 B]
Get:27 http://ports.ubuntu.com/ubuntu-ports jammy-backports/universe arm64 Packages [31.9 kB]
Get:28 http://ports.ubuntu.com/ubuntu-ports jammy-backports/universe Translation-en [16.9 kB]
Get:29 http://ports.ubuntu.com/ubuntu-ports jammy-backports/universe arm64 c-n-f Metadata [604 B]
Get:30 http://ports.ubuntu.com/ubuntu-ports jammy-backports/multiverse arm64 c-n-f Metadata [116 B]
Get:31 http://ports.ubuntu.com/ubuntu-ports jammy-security/main arm64 Packages [3032 kB]
Get:32 http://ports.ubuntu.com/ubuntu-ports jammy-security/main Translation-en [448 kB]
Get:33 http://ports.ubuntu.com/ubuntu-ports jammy-security/main arm64 c-n-f Metadata [13.9 kB]
Get:34 http://ports.ubuntu.com/ubuntu-ports jammy-security/restricted arm64 Packages [5612 kB]
Get:35 http://ports.ubuntu.com/ubuntu-ports jammy-security/restricted Translation-en [1070 kB]
Get:36 http://ports.ubuntu.com/ubuntu-ports jammy-security/restricted arm64 c-n-f Metadata [528 B]
Get:37 http://ports.ubuntu.com/ubuntu-ports jammy-security/universe arm64 Packages [1082 kB]
Get:38 http://ports.ubuntu.com/ubuntu-ports jammy-security/universe Translation-en [226 kB]
Get:39 http://ports.ubuntu.com/ubuntu-ports jammy-security/universe arm64 c-n-f Metadata [20.2 kB]
Get:40 http://ports.ubuntu.com/ubuntu-ports jammy-security/multiverse arm64 Packages [34.6 kB]
Get:41 http://ports.ubuntu.com/ubuntu-ports jammy-security/multiverse Translation-en [10.5 kB]
Get:42 http://ports.ubuntu.com/ubuntu-ports jammy-security/multiverse arm64 c-n-f Metadata [308 B]
93% [12 Translation-en store 0 B]                                                                    995 kB/s 0s
```

Step 7: Install Nginx inside VM

```
sudo apt update
sudo apt install nginx
sudo systemctl start nginx
```

Step 8: Verify Nginx in VM

```
curl localhost
```

```
</html>
vagrant@ubuntu:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
vagrant@ubuntu:~$ exit
```

Step 9: Observe Resource Usage (VM)

CPU and memory usage were monitored using Task Manager while VM was running.

```
pratiksmac@MacBook-3 Experiment-1 % vagrant halt
vagrant destroy
==> default: Stopping the instance...
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Stopping the instance...
==> default: Destroying the instance...
pratiksmac@MacBook-3 Experiment-1 %
```

Part B: Container Setup (Docker)

Step 10: Pull Ubuntu and Nginx Images

```
docker pull ubuntu
docker pull nginx
```

Step 11: Run Nginx Container

```
docker run -d -p 8080:80 --name nginx-container nginx
```

Step 12: Verify Nginx in Container

```
curl localhost:8080
```

Commands Used

```
vagrant init ubuntu/jammy64
vagrant up
vagrant ssh
sudo apt update
sudo apt install nginx
curl localhost
vagrant halt
vagrant destroy

docker pull ubuntu
docker pull nginx
docker run -d -p 8080:80 --name nginx-container nginx
docker images
```

Observation

It was observed that the Virtual Machine consumed significantly higher CPU and memory resources because it runs a complete operating system. The Docker container started faster and consumed fewer resources while running the same Nginx service.

Conclusion

Containers are more efficient than Virtual Machines. Containers are lightweight, start faster, and consume fewer system resources because they share the host operating system kernel. This experiment clearly demonstrates that container-based virtualization is better suited for modern application deployment compared to traditional Virtual Machines.